Dalibor Nasevic
Software Engineer at GoDaddy working on the email delivery infrastructure.

-
-
-
-
-

Archives

- 2017 (2)
- 2016 (13)
- 2015 (13)
- 2014 (8)
- 2013 (5)
- 2012 (5)
- 2011 (17)
- 2010 (7)
- 2009 (7)

Permanently remove files and folders from Git repo

> Note: In this blog post the operations that are presented will override git history. Be careful what you're doing and backup your repo if you're not sure what you're doing.

Few weeks ago I froze gems on my blog and ended up with a very big repository. So, I wanted to clean up the mess and remove permanently gems folder from the repository. `git rm` wasn't doing the job well, it only removes the folder from the working tree and the repository still contains the objects of this folder. After a quick search, I found that git-filter-branch was the command I was looking for.

So, you can permanently remove a folder from a git repository with:

```
git filter-branch --tree-filter 'rm -rf vendor/gems' HEAD
```

Which will go through the whole commits history in the repository, one by one change the commit objects and rewrite the entire tree.

We use -r (recursive) parameter for recursive remove, and -f (force) to ignore nonexistent files (since folder/files may not be introduced to the repository within the commits range on which we do branch filter).

You can also specify range between commits, where you like to filter:

```
git filter-branch --tree-filter 'rm -rf vendor/gems' 7b3072c..HEAD
```

First commit is not being filtered.

If you subsequently try to do branch filters, you should provide -f option to filter-branch to overwrite the backup in refs/original/ where git stores the original refs from the previous branch filter.

```
git filter-branch -f --tree-filter 'rm -rf vendor/gems' HEAD
```

You can also remove original refs by hand, or do some backup to other location.

```
rm -rf .git/refs/original/
```

Permanently removing files from repository is same as folders:

```
git filter-branch --tree-filter 'rm filename' HEAD
```

There are few branch filter types (you can check the documentation), but the one we use here --tree-filter is for rewriting the tree and its contents. You can also use --index-filter which is similar to --tree-filter but does not check the tree, and it goes much faster.

```
git filter-branch --index-filter 'git rm --cached --ignore-unmatch filename' HEAD
```

`--ignore-unmatch` parameter is used to ignore nonexistent files.

At the end, don't forget to push the changes to the repository with --force, since this is not a fast forward commit, and the whole history within the commits range we filtered will be rewritten.

```
git push origin master --force
```

Update: While `filter-branch` rewrites the history for you, the objects remain in your local repository until they get dereferenced and garbage collected.

To check what's pointing to nuked objects with use the following command. If you have tags and branches in the repo pointing to those objects, you'll most likely see them.

```
git for-each-ref --format='delete %(refname)' refs/original
```

To dereference, expire reflog (which by default is 90 days) and force garbage collect, you can do:

```
git for-each-ref --format='delete %(refname)' refs/original | git update-ref --stdin
git reflog expire --expire=now --all
git gc --prune=now
```

You'll need to make sure all branches and tags are pushed to remote (unless you're pushing to a new repo).

```
git push origin --force --all
git push origin --force --tags
```

Posted by Dalibor Nasevic on July 07, 2009 and tagged with: git.

45 Comments Dalibor Nasevic 1 Login

Recommend 13 Share Sort by Best

Join the discussion…

bARTosz • 5 years ago

Recursively remove all files of filetype .foo from the repo:

git filter-branch --tree-filter 'git rm -r -f --ignore-unmatch *.pyc' HEAD

11 • Reply • Share ›

Kory Mathewson → bARTosz • a year ago

Thank you kindly!

• Reply • Share ›

Rajat • 5 years ago

Got one tiny doubt, ran the git filter command in my local and remove the file. Now in order to remove the files from repository, do I need to do a git push??
Please help.

Regards,
Rajat

9 • Reply • Share ›

jim ➦ Rajat • a year ago

ever figure this one out?

• Reply • Share ›

Brad ➦ jim • a year ago

As the article says:

> At the end, don't forget to push the changes to the repository with --force, since this is not a fast forward commit, and the whole history within the commits range we filtered will be rewritten.

`git push origin master --force`

• Reply • Share ›

mjvfx • 3 years ago

Recursively remove multiple complete directories /foo from the repo:

git filter-branch --tree-filter 'git rm -r -f --ignore-unmatch */foo/* ' HEAD

3 • Reply • Share ›

Bicz • 4 years ago

Thank you, you saved me.

1 • Reply • Share ›

Frank Grießhammer • 5 months ago

I have found the following discoveries to be useful:

– This fact might be obvious to some, but it wasn't for me:
A range can only be specified from hash to HEAD (not hash-to-hash)

> `--prune-empty`

will get rid of empty commits which result from folder deletion

> `--tag-name-filter cat`

will re-assign tags to rewritten commits

The command I ran:

```
git filter-branch -f --tree-filter 'rm -rf folderName' --prune-empty --tag-name-filter
```

Reply • Share ›

Lucian Pacurar • 5 months ago

Great article! I have one question though: I have a dozen branches created from master, some of which still live. What happens if I rewrite the entire git history like this and then push it to the remote? Will this break all the other branches?

Reply • Share ›

Dalibor Nasevic Mod → Lucian Pacurar • 5 months ago

It shouldn't break the other branches but you will not be able to reclaim the space because other branches will still reference the objects you try to get rid of on your branch.

Reply • Share ›

Lucian Pacurar → Dalibor Nasevic • 5 months ago

But if I fetch all branches from remote, then "-- --all" should do it, right? Thanks :)

Reply • Share ›

Dalibor Nasevic Mod → Lucian Pacurar • 5 months ago

I'm not sure what that will do, but to reclaim the space you'll need to make sure no branches reference the objects you're removing, so either you'll need to force push or delete those branches from local and remote.

Reply • Share ›

Jürgen Steinblock • 6 months ago

Thank you, saved me ~100MB in my repository.

After filter-branch look for the outcome.

On the first run I had a type and the output was: "WARNING: Ref 'refs/heads/master' is unchanged."
Solved the type and got "Ref 'refs/heads/master' was rewritten"

Reply • Share ›

Jack Thomas Lange → Jürgen Steinblock • a month ago

What do you mean by "had a type"?

Do you mean "typo"? I'm having the same issue

Reply • Share ›

Jürgen Steinblock ➦ Jack Thomas Lange • a month ago

yes, I ment typo :-)

That basically means that, after applying a filter that does not do anything i.e.

git filter-branch -f --tree-filter 'rm -rf vendoor/gems' HEAD

the repo did not change. In other words, recheck your rm command and also check what happens if you just execute "rm -rf vendor/gems" on a brach.

Reply • Share ›

Bret Hart • 8 months ago

Very useful. thanks a million.

Reply • Share ›

Ramesh • 10 months ago

after rewrite history git repo size increased

i ve tried the below commands

```
git clone <repo>
cd <repo>
git filter-branch --tag-name-filter cat --index-filter 'git rm -r --cached --ignore-unmatch XXX' --prune-empty -f -- --all
rm -rf .git/refs/original/
git reflog expire --all
git gc --aggressive --prune=now

git reflog expire --all --expire-unreachable=0
git repack -A -d
git prune

git push --all --force
```

After all repo size were double the time increased and this chnages got reflected only master repo

please guide me what to do now ?

Reply • Share ›

Marine Bocquet • a year ago

It keep asking me for my github pass to download the history before modifying it, it's very annoying...

Reply • Share ›

Kyle ➦ Marine Bocquet • a year ago

Here you are my friend http://stackoverflow.com/qu...

Reply • Share ›

Rich Cook • a year ago

I just used this to remove all files larger than 1MB from my repo: git filter-branch --tree-filter 'find ./ -size +1M -delete' HEAD

Reply • Share ›

Cap'n Slipp • a year ago

The git-filter-branch man page recommends (under the Checklist for Shrinking a Repository section) using the command:

```
git for-each-ref --format="%(refname)" refs/original/ |
        xargs -n 1 git update-ref -d
```

to remove the original refs, as opposed to (as you've written) deleting the repo's ref-tracking files using the OS-level filesystem tool:

```
rm -rf .git/refs/original/
```

Other than that, useful info here; I keep coming back to your post as a cheatsheet when I need to remember this stuff. Cheers.

Reply • Share ›

qwerty • a year ago

how to delete a specific file from Git Repository

Reply • Share ›

Hao • 2 years ago

Great blog. Though it requires really some knowledge about git to get everything. The command about dereference, you can actually leave out that final "--stdin".

Reply • Share ›

An Luutran • 2 years ago

My understanding is that once the files or folders get saved into the .git/objects store, they

My understanding is that once the files or folders get saved into the .git/objects store, they remained there permanently. So, the command suggested may give the impression that the files or folders are no longer there, but in fact they can always be retrieved given their commit ID if it can be recalled. What this means is that git is 100% secure in terms of data protection (barring file system/hard drive corruption, or forcefully using UNIX 'rm' cmd) and thus the space taken by the files/folders can't be reclaimed/freed up. Thus, a mistaken 'git push --force' is not disastrous as it can be undone if prior to doing that the "backup" commit ID is recorded somewhere.

Reply • Share ›

Dalibor Nasevic Mod → An Luutran • 2 years ago

That's a good point and I've updated the blog post with how to dereference and force expire the reflog if you need/want.

Reply • Share ›

nimo • 2 years ago

Thank you

Reply • Share ›

Ilter Cengiz • 2 years ago

After almost 5 years, this is still valid and helpful.
Thank you for sharing!

Reply • Share ›

Phantomfav • 2 years ago

I also recommend the --all option at the end of the command to include every branch present in the repository.

Reply • Share ›

Diogo Pessoa • 2 years ago

Thank you, just had to make a clean up and this helped a lot!!

Reply • Share ›

gggeek • 3 years ago

A question, before I spend a couple of days testing: is the filter-branch command usable in the following scenario?
- local repo is clone of an upstream one
- the upstream one has other clones as well
- while I rewrite history on the local clone, someone pushes new commits to the upstream repo
Will I be able to do a pull+push --force after rewriting history?

Will the other clones be able to keep pushing new commits upstream after upstream has got a rewritten history?

Reply • Share ›

Dalibor Nasevic Mod → gggeek • 3 years ago

Unfortunately, no. If you force push your overridden git history, you'll break the repo for your collaborators.

Reply • Share ›

John Koisch • 3 years ago

Nice article. One thing that helped me ... I had to put the path in '"' quotations.

Reply • Share ›

Marco Franssen • 4 years ago

It seems the first command does not work for me. I tried to remove some folders (including all content) from my repository permanently. However after the complete tree is rewritten it tells me nothing has changed. Any ideas on how to solve this? I used a relative path in the command just as in your example. I am in a develop branch.

I hope you can help me with this.

Reply • Share ›

nebev • 4 years ago

Worked a treat. Thanks

Reply • Share ›

Sergey Tsivin • 4 years ago

Thanks a lot! It worked like a charm.

Reply • Share ›

Andrew Mao • 4 years ago

Awesome, super helpful, thanks! I found your post a useful complement to this SO post about moving stuff from one repo to another: http://stackoverflow.com/qu...

Reply • Share ›

A legnagyobb prím szám • 5 years ago

Thanks for this post! Googling "permanently remove git" took me here and saved me from going through the whole GitPro book (which I still do recommend, it's worth).

Just a few questions / additions:

just a few questions / additions:

- Another related post mentions that doing 'git gc' is important before the branch-filter so I did. What are your thoughts on this? Would branch-filter complain and not start or would it not have the desired results w/o gc beforehand?

- I found that in case you work with multiple clone's of a repo (e.g. one on your dev box, another on a test server and yet another for the live version of your app, then it's important to bring all the other working copies into sync with the repo (i.e. pull) before you do filter-branch. Otherwise, (if you have committed and pushed all changes from your development working copy that aren't pulled into the others) when you pull after the filter-branch on the other boxes, you won't have a "clean" pull and will end up with conflicts. That will result in having to merge and push back from those which will lead to having to pull again from where you did the filter-branch and you will lose the changes of the filter-branch (or they will be overwritten). At least that's what happened to me and I had to redo the filter-branch, because the permanently removed content came back. I'm not a git pro and don't have a complete understanding of what happened, so I'd be glad if you had something to add to this.

Regards,Marcel

Reply • Share ›

Dalibor Nasevic Mod → A legnagyobb prím szám • 5 years ago

git filter-tree is rewriting the commits tree, so all other clones of the repo will be out of sync (they will need to clone from the original repo again). You need to think if it's safe for you to run this command before running it.

git gc is just deleting loose objects and packing objects to use disk space more efficiently (so it's not needed for this operation).

1 Reply • Share ›

A legnagyobb prím szám → Dalibor Nasevic • 5 years ago

Thanks for your answer. OK, I have to read more of the GitPro book. It's not very clear at the moment to me what exactly is the difference between index-filter and tree-filter type of filter-branch operations. Am I right supposing that rewriting the entire commit tree (tree-filter) will result in more efficiently shrinking the size of the repository removing committed objects while index-filter will only remove metadata from it (commit history entries, logs, etc. but leave the actual committed files)?
My goal is not have large sized unnecessary data in the repository. I don't really care about a few tiny files that became obsolete, but in this particular case when I started looking into using the filter-branch command, I took over a project which had significant amount of files committed to its repo.

I'd appreciate if you could share your experience of this aspect.

Kind Regards,
Marcel

Reply • Share ›

vikash vikram • 5 years ago

Hi,
I acidently added a file and pushed it to git. I removed it from my application and by using git rm command also. but it still persists in .git directory (it is a movie file so makes its presence very obvious). when i use filter branch to permanently delete the file, it says no such file or directly. Any ideas on what should i do ???

Reply • Share ›

kostadin • 8 years ago

Fala Dalibore za ovoj post, me spasi od katastrofa... Dobar blog isto taka-bev iznenaden da vidam vakov tip na (kvaliteten) blog da ide od mak.

Nazdravje!

Reply • Share ›

DRB • 8 years ago

Thank you!

I also read the following but found your solution the easiest to use:
http://github.com/guides/co...
http://www.gitready.com/beg...

Reply • Share ›

Tim Swinson • 10 months ago

Wonderful! So simple. I am sure there are many people who are faced with the same problems I recently had. I couldn't find an online service for merging PDF files, but eventually I found a simple one. I've found some decent tutorials on how to merge files out online here `http://goo.gl/Sz6wLb`

Reply • Share ›

Scritti Politti • 5 years ago

Thanks for the guide. But my results have not been good. I had an up-to-date local repository, everything committed and pushed to the server. Then I ran this:

git filter-branch --tree-filter 'rm filename' HEAD

Git complained that there was a backup in refs/original, and that I had to use the -f flag to overwrite it. So I did. The result is this:

On branch master
Your branch and 'origin/master' have diverged,
and have 57 and 57 different commit(s) each, respectively.

Now I have no idea how to get these back in sync.

UPDATE: I found instructions saying to issue:

git remote update

and this seems to have worked. The question is why the branches were seen as having diverged in the first place.

see more

Reply • Share ›

Dalibor Nasevic Mod → Scritti Politti • 5 years ago

"Which will go through the whole commits history in the repository, one by one change the commit objects and rewrite the entire tree." - as it says, this operation with rewrite all your commits

If the repo has more than one commiter it might be a problem because they need to clone the new repo after you push with git push origin master -f. -f means it will overwrite the main repo with the new commits, but then all commiters that have cloned the repo with have a different tree and will need to clone the repo again. So again, this operation is destructive, be careful with this kind of operations and always do a backup first if you don't know what you are doing.

Reply • Share ›

ALSO ON DALIBOR NASEVIC

Deploying Rails applications

2 comments • 4 years ago•

Dalibor Nasevic — Yes, you can specify local/remote branches for each stage with:c.local_branch = …

12 Vim Tips

12 comments • 3 years ago•

buztard — The :args command also supports external commands using backticks.:args `ag -l CurrencyNumberHelper`:argdo …

Ruby class_eval, __FILE__ and __LINE__ arguments

Intro to Machine Learning in Ruby

2 comments • a year ago•

2 comments • 5 years ago•

solife — heredoc, brackets usage...thanks.https://github.com/rails/ra...

Dalibor Nasevic — News aggregators became popular at the time and this was an experiment / show-off project to see what's possible to …

Subscribe Add Disqus to your siteAdd DisqusAdd Privacy DISQUS